BUILDING STATISTICAL MODELS

OPTICAL CHARACTER RECOGNITION

12

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Exemplo de detecção de texto em imagem 5
Figura 2 – Exemplo de reconhecimento de texto em imagem 6
Figura 3 – Exemplo de pipeline de reconhecimento de texto em imagem 7
Figura 4 – CRNN - estrutura.....10
Figura 5 – download modelo DB_TD500_resnet50.onnx.....11
Figura 6 – Exemplos de resultados de detecção de texto.....16
Figura 7 – download modelo de reconhecimento CRNN 17
Figura 6 – Exemplos de resultados de reconhecimento de texto.....30
Figura 9 – Arquitetura de detecção PP-OCR 31
Figura 10 – Reconhecimento de texto PaddleOCR.....34
Figura 11 – Optimus Prime e Bumble Bee; Transformers na Universal Studios 35
Figura 12 – Arquitetura open Clip.....38
Figura 12 – lista de modelos pré-treinados 39
Figura 13 – exemplo de resultado de OCR com Transformers 42
Figura 15 – Arquitetura TrOCR 45
Figura 16 – Reconhecimento TrOCR Hugging Face.....49
Figura 17 – Hugging Face - TrOCR 50
Figura 18 – Notícia jornal - Senna.....52

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Inicializa o modelo da rede 13
Código-fonte 2 – pré-processamento da imagem 13
Código-fonte 3 – ajustas as predições realizadas 14
Código-fonte 4 – ajustas as predições realizadas 16
Código-fonte 5 – Detecta faces 18
Código-fonte 6 – função visualize 20
Código-fonte 7 – Função initialize_english_textrecognition_model 21
Código-fonte 8 – Reconhece texto 22
Código-fonte 9 – Função main de reconhecimento de texto 24
Código-fonte 10 – Código completo de reconhecimento de texto 29
Código-fonte 11 – Infra PaddleOCR 32
Código-fonte 12 – Reconhecimento OCR com PaddleOCR 33
Código-fonte 13 – Instala biblioteca openclip e transformers 38
Código-fonte 14 – list pretrained 38
Código-fonte 15 – Carrega modelo 40
Código-fonte 16 – Carrega imagem 40
Código-fonte 17 – gera predição de texto 40
Código-fonte 18 – lista de camisetas 42
Código-fonte 19 – instala bibliotecas 45
Código-fonte 20 – Código TrOCR 48

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

OPTICAL CHARACTER RECOGNITION 5

1 INTRODUÇÃO 5

1.1 Pipeline Básico de Detecção e reconhecimento de Texto 6

2 DETECTANDO E RECONHECENDO TEXTO COM OPENCV 9

2.1 Detecção de texto 11

2.2 Reconhecimento de texto 17

2.3 Conhecendo o PPOCR 30

3 GENERATIVE IA EM OCR 35

3.1 Vision Transformers 36

3.2 TrOCR 43

4 HANDS ON: PRATICANDO O RECONHECIMENTO DE CARACTERES DE TEXTO 51

4.1 Aquecimento 51

4.2 Seu desafio 51

4.3 Dicas de solução 52

REFERÊNCIAS 54

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

OPTICAL CHARACTER RECOGNITION

1 INTRODUÇÃO

A detecção e o reconhecimento de texto são duas das diversas aplicações da visão computacional, com diversas utilidades, incluindo análise de documentos, como um scanner digital ou o reconhecimento de placas de veículos por radares de velocidade.

Figura 1 – Exemplo de detecção de texto em imagem
Fonte: autor

A detecção de texto envolve localizar e identificar conteúdo textual dentro de imagens ou vídeos. O objetivo principal é identificar as regiões onde o texto está presente, sendo um passo crucial de pré-processamento para o reconhecimento de texto. Por sua vez, o reconhecimento de texto, também conhecido como Reconhecimento Óptico de Caracteres (OCR), é o processo de converter o texto detectado dentro de uma imagem em texto legível por máquina e editável. O objetivo é entender e transcrever o conteúdo textual com precisão. O OCR pode ser realizado usando técnicas tradicionais, como correspondência de templates e reconhecimento de padrões, mas os sistemas mais utilizados de OCR geralmente dependem de modelos de deep learning.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 2 – Exemplo de reconhecimento de texto em imagem
Fonte: autor

Combinar a detecção e o reconhecimento de texto permite que os sistemas de visão computacional localizem e decifrem o texto dentro das imagens, tornando as informações textuais acessíveis para processamento e análise posterior.

Os desafios tanto na detecção como no reconhecimento de texto são diversos e incluem lidar com texto multi-orientado (vertical, horizontal, diagonal), detectar texto em fundos complexos assim como diferenciar entre regiões de texto e não-texto, além de lidar com variações em fontes, tamanhos, idiomas e distorções devido à perspectiva, iluminação ou ruído. Enfim, trata-se de uma atividade não trivial e que envolve aplicação de diversos conceitos para que sejam possíveis aplicações na digitalização de documentos impressos e extração de informações de imagens.

1.1 Pipeline Básico de Detecção e reconhecimento de Texto

A detecção e o reconhecimento de texto em imagens envolvem várias etapas sequenciais que, em conjunto, formam um pipeline básico. Esse pipeline pode variar dependendo da complexidade da aplicação e das técnicas utilizadas, mas geralmente segue os seguintes passos:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

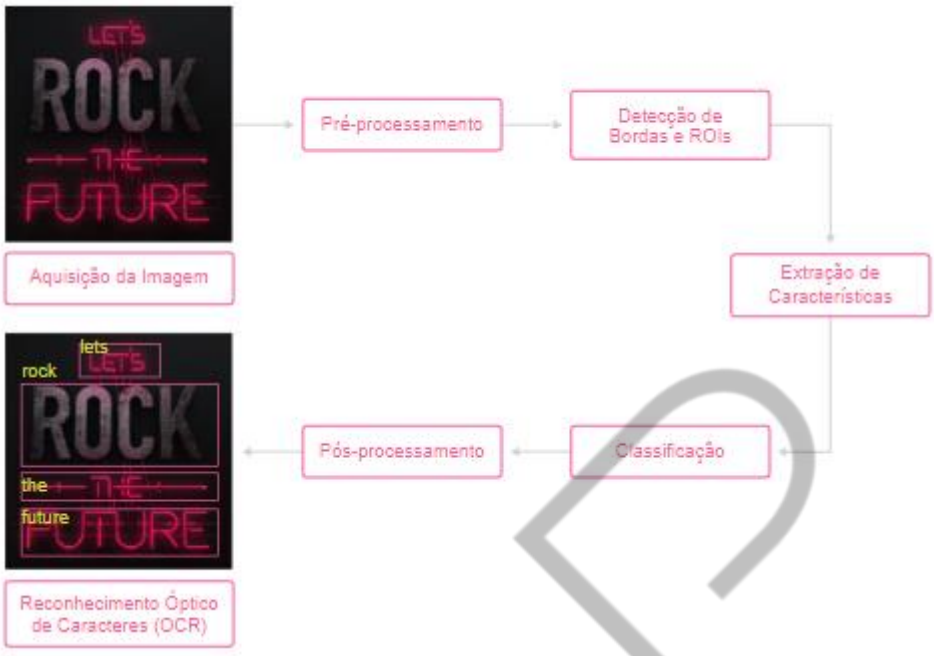

Figura 3 – Exemplo de pipeline de reconhecimento de texto em imagem
Fonte: autor

- **Aquisição da Imagem**: o primeiro passo é adquirir a imagem na qual o texto será detectado. As fontes dessas imagens podem variar, incluindo câmeras, scanners ou bases de dados de imagens digitais. A qualidade e a resolução da imagem podem afetar significativamente o desempenho da detecção de texto;
- **Pré-processamento**: o pré-processamento é crucial para preparar a imagem para a detecção de texto. As técnicas comuns de pré-processamento incluem: Redimensionamento; Conversão para Escala de Cinza; Filtragem (reduzir ruído e melhorar a qualidade da imagem); Equalização do Histograma (Melhorar o contraste), dentre outras, como binarização e normalização;
- **Detecção de Bordas e Segmentação de Regiões de Interesse (ROIs)**: essa é uma etapa importante no pipeline de detecção de texto, pois as bordas podem ajudar a identificar regiões de interesse que possivelmente contêm texto;
- **Extração de Características**: essa etapa envolve identificar e extrair atributos significativos das regiões de interesse. Esses atributos ajudam a diferenciar entre texto e não-texto, e é comum utilizar técnicas como

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Histogramas de Gradientes Orientados (HOG) ou Descritores de Textura para identificar padrões de textura que são característicos de áreas de texto;

- **Classificação**: essa etapa utiliza modelos de aprendizado de máquina ou redes neurais para identificar se as regiões de interesse contêm texto como SVM ou **CNNs**. Podemos treinar nossos próprios modelos ou utilizar modelos específicos para detecção de texto que são pré-treinados em grandes bases de dados de texto e oferecem alta precisão na detecção, como o **EAST** (Efficient and Accurate Scene Text Detector), **DB** (Differentiable Binarization), CTPN (Connectionist Text Proposal Network) ou CRAFT (Character Region Awareness for Text detection), dentre outras;
- **Pós-processamento**: é a etapa final, onde os resultados da **detecção** são refinados eliminando bound boxes redundantes das regiões de texto detectadas;
- **Reconhecimento Óptico de Caracteres (OCR):** após a detecção de texto, pode-se aplicar um sistema de reconhecimento óptico de caracteres (OCR) para converter as imagens de texto em texto editável. Podemos utilizar ferramentas como o **Tesseract,** que é uma biblioteca, ou sistemas baseados em deep learning, como **CRNN** (Convolutional Recurrent Neural Network), que podem ser treinadas do zero ou pré-treinadas.

Esse pipeline fornece apenas uma visão geral e pode variar conforme a aplicação, mas, de qualquer forma, nos ajuda a formar uma compreensão mais concreta de como é realizado o processo de reconhecimento de texto em imagens e que não se trata de uma tarefa tão simples. Que tal partir para a prática e começar a implementar essas etapas? É isso que veremos na próxima seção.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 DETECTANDO E RECONHECENDO TEXTO COM OPENCV

A OpenCV possui um módulo chamado DNN (Deep Neural Network), que permite a utilização de redes neurais pré-treinadas para realizar diversas tarefas, incluindo a detecção de texto. Essas redes neurais foram treinadas em grandes conjuntos de dados de imagens e textos, o que lhes permite identificar padrões e caracteres com alta precisão.

No link https://docs.opencv.org/4.x/d4/d43/tutorial_dnn_text_spotting.html, a documentação disponibiliza alguns modelos para essa tarefa. Note que existem modelos de detecção e de reconhecimento de texto; cada modelo com suas particularidades.

Modelos de Detecção de Texto:

- DB (Differentiable Binarization): um modelo que utiliza binarização diferenciável para detectar texto em imagens com baixa qualidade;
- EAST (Efficient and Accurate Scene Text Detector): um modelo eficiente e preciso para detectar texto em imagens de cena complexas.

Ambos os métodos, DB e EAST, têm seus pontos fortes. O DB é particularmente eficaz em situações em que a precisão de binarização é crucial, enquanto o EAST se destaca pela sua simplicidade e eficiência em tempo real. A escolha entre esses métodos depende dos requisitos específicos da aplicação, como a necessidade de precisão versus velocidade.

Saiba mais: Para ter mais informações sobre as técnicas DB e EAST, acesse os artigos sobre cada uma delas:

EAST: https://paperswithcode.com/paper/east-an-efficient-and-accurate-scene-text?ref=blog.roboflow.com

BD: https://arxiv.org/abs/1911.08947?ref=blog.roboflow.com

Modelos de Reconhecimento de Texto:

- CRNN (Convolutional Recurrent Neural Network): trata-se de um modelo que combina as propriedades de redes neurais convolucionais (CNNs) e redes neurais recorrentes (RNNs) para realizar a tarefa de

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

reconhecimento de texto em imagens. Esse modelo é particularmente eficaz para OCR, pois pode lidar com sequências de caracteres de comprimento variável.

A figura a seguir descreve a arquitetura da rede neural, que consiste em três partes:

- Camadas Convolucionais: extrai características visuais da imagem. Essas camadas são similares às utilizadas em CNNs tradicionais e são responsáveis por capturar padrões locais da imagem;
- Camadas Recorrentes: predizem a distribuição de rótulos para cada quadro da sequência;
- Camada de Transcrição: traduz as previsões por quadro na sequência final de rótulos. Essa camada utiliza a técnica de Connectionist Temporal Classification (CTC) para alinhar as previsões de rótulos aos quadros da sequência de características, permitindo a geração da sequência de texto final sem a necessidade de segmentação explícita dos caracteres na imagem de entrada.

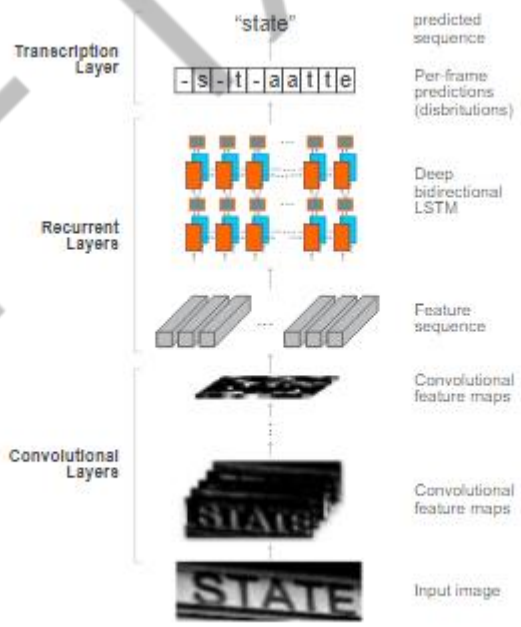

Figura 4 – CRNN - estrutura
Fonte: Shi (2015)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Saiba mais: Para ter mais informações sobre CRNN, acesse o artigo:

CRNN: https://arxiv.org/pdf/1507.05717

A seguir, vamos detalhar como utilizar um desses modelos para detecção de texto e outro para reconhecimento de texto.

2.1 Detecção de texto

Vamos implementar o nosso detector de texto a partir de imagens. Para isso, utilizaremos o modelo DB (Differentiable Binarization) Text Detection.

O primeiro passo é realizar o download dos pesos da rede pré-treinada 'DB_TD500_resnet50.onnx'. Fazemos isso a partir do link disponibilizado pela própria documentação da OpenCV https://drive.google.com/uc?export=dowload&id=19YWhArrNccaoSza0CfkXlA8im4-lAGsR, como demonstra a figura a seguir.

TextDetectionModel

```
- DB_IC15_resnet50.onnx:
url: https://drive.google.com/uc?export=dowload&id=17_ABp79PlFt9yPCxSaarVc_DKTmrSGGf
sha: bef233c28947ef6ec8c663d20a2b326302421fa3
recommended parameter setting: -inputHeight=736, -inputWidth=1280;
description: This model is trained on ICDAR2015, so it can only detect English text instances.

- DB_IC15_resnet18.onnx:
url: https://drive.google.com/uc?export=dowload&id=1vY_KsDZZZb_svd5RT6pjyI8BS1nPbBSX
sha: 19543ce09b2efd35f49705c235cc46d0e22df30b
recommended parameter setting: -inputHeight=736, -inputWidth=1280;
description: This model is trained on ICDAR2015, so it can only detect English text instances.

- DB_TD500_resnet50.onnx:
url: https://drive.google.com/uc?export=dowload&id=19YWhArrNccaoSza0CfkXlA8im4-lAGsR
sha: 1b4dd21a6baa5e3523156776970895bd3db6960a
recommended parameter setting: -inputHeight=736, -inputWidth=736;
description: This model is trained on MSRA-TD500, so it can detect both English and Chinese text instances.

- DB_TD500_resnet18.onnx:
url: https://drive.google.com/uc?export=dowload&id=1sZszH3pEt8hliyBlTmB-iulxHP1dCQWV
sha: 8a3700bdc13e00336a815fc7afff5dcc1ce08546
recommended parameter setting: -inputHeight=736, -inputWidth=736;
description: This model is trained on MSRA-TD500, so it can detect both English and Chinese text instances.
```

We will release more models of DB here in the future.

```
- EAST:
Download link: https://www.dropbox.com/s/r2ingd0l3zt8hxs/frozen_east_text_detection.tar.gz?dl=1
This model is based on https://github.com/argman/EAST
```

Images for Testing

Figura 5 – download modelo DB_TD500_resnet50.onnx
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Com os pesos já baixados, podemos dar continuidade e começar o nosso código. Vamos criar algumas funções para facilitar o entendimento e legibilidade do código. Começamos carregando o Numpy e a OpenCV e criando uma função 'initialize_model' para inicializar o nosso modelo. A função recebe como parâmetro o modelo pré-treinado e o shape da imagem da rede. Além disso, outros valores já são pré-definidos para a configuração do modelo, esses valores podem ser ajustados caso seja necessário um ajuste fino na etapa de detecção.

```
import numpy as np
import cv2

def initialize_model(model_path, model_shape, binary_threshold=0.3,
polygon_threshold=0.5,
                     max_candidates=200, unclip_ratio=2.0,
scale=1.0/255.0, mean=(122.67891434, 116.66876762, 104.00698793)):
    """
    Inicializa o modelo de detecção de texto.

    Parâmetros:
    - model_path: Caminho para o modelo pré-treinado.
    - model_shape: Tamanho da imagem de entrada (largura, altura).
    - binary_threshold: Limiar para a binarização.
    - polygon_threshold: Limiar para a formação do polígono.
    - max_candidates: Número máximo de candidatos (caixas de texto).
    - unclip_ratio: Fator de desenclipe para expandir as caixas.
    - scale: Fator de escala para normalização dos pixels.
    - mean: Média para normalização dos pixels.

    Retorna:
    - O modelo de detecção de texto inicializado.
    """

    # Carrega o modelo
    model = cv2.dnn_TextDetectionModel_DB(cv2.dnn.readNet(model_path))

    # Configurações do modelo
    model.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
    model.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
    model.setBinaryThreshold(binary_threshold)
    model.setPolygonThreshold(polygon_threshold)
    model.setUnclipRatio(unclip_ratio)
    model.setMaxCandidates(max_candidates)
    model.setInputParams(scale, model_shape, mean)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    return model
```

Código-fonte 1 – Inicializa o modelo da rede
Fonte: autor

Agora, vamos criar a função ‘preprocess_image’, que é responsável por carregar e redimensionar a imagem de entrada para o tamanho que a rede neural que estamos utilizando espera como entrada. Além disso, retornamos o fator de escala da imagem. Esse passo é realizado passando como argumentos da função: image_path, que corresponde ao caminho de diretórios em que a imagem está armazenada, e model_shape, que representa o shape (dimensional) da rede neural que estamos utilizando.

```
def preprocess_image(image_path, model_shape):
    """
    Carrega e redimensiona a imagem de entrada para o tamanho do
modelo.

    Parâmetros:
    - image_path: Caminho para a imagem de entrada.
    - model_shape: Tamanho da imagem de entrada (largura, altura).

    Retorna:
    - original_image: A imagem original carregada.
    - image: A imagem redimensionada para o tamanho do modelo.
    - scaleWidth: Fator de escala para a largura.
    - scaleHeight: Fator de escala para a altura.
    """
    original_image = cv2.imread(image_path)
    original_h, original_w, _ = original_image.shape
    scaleHeight = original_h / model_shape[1]
    scaleWidth = original_w / model_shape[0]
    image = cv2.resize(original_image, model_shape)

    return original_image, image, scaleWidth, scaleheight
```

Código-fonte 2 – pré-processamento da imagem
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Com o modelo configurado e a imagem corretamente carregada, podemos realizar as detecções de texto. O método ‘detect’ realiza as predições e retorna uma lista de 2 elementos, em que o elemento [0] contém os polígonos das regiões de texto, com as coordenadas normalizadas, e o elemento [1] contém as probabilidades associadas a cada região de texto. Para processar esses resultados, vamos criar a função ‘adjust_results’, que recebe o vetor de resultados e o fator de escala da imagem de entrada e retorna os resultados ajustados para a escala original.

Cada polígono é uma lista de 4 pontos (x, y), por essa razão, criamos um laço for encadeado para varrer todos os pontos de interesse:

- results[0][i][j] acessa o j-ésimo ponto do i-ésimo polígono;
- results[0][i][j][0] acessa a coordenada x do j-ésimo ponto do i-ésimo polígono;
- results[0][i][j][1] acessa a coordenada y do j-ésimo ponto do i-ésimo polígono.

```
def adjust_results(results, scaleWidth, scaleHeight):
    """
    Ajusta as coordenadas das caixas de resultados de volta para a
escala original da imagem.

    Parâmetros:
    - results: Resultados da detecção, contendo os polígonos das
regiões de texto.
    - scaleWidth: Fator de escala para a largura.
    - scaleHeight: Fator de escala para a altura.

    Retorna:
    - results: Resultados ajustados para a escala original.
    """
    for i in range(len(results[0])):
        for j in range(4):
            box = results[0][i][j]
            results[0][i][j][0] = box[0] * scaleWidth
            results[0][i][j][1] = box[1] * scaleheight

    return results
```

Código-fonte 3 – ajustas as predições realizadas

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: autor

Pronto, a parte mais complicada já está concluída! Agora, podemos finalizar nosso código de detecção criando as funções ‘visualize’ e ‘main’ para desenhar os bound boxes preditos e executar o nosso código.

```
def visualize(image, results):
    """
    Desenha os polígonos dos resultados da detecção na imagem.

    Parâmetros:
    - image: Imagem original onde os resultados serão desenhados.
    - results: Resultados da detecção, contendo os polígonos das
regiões de texto.

    Retorna:
    - A imagem com os polígonos desenhados.
    """

    box_color = (0, 255, 255)  # Cor do box (amarelo)
    isClosed = True  # Indica se o polígono é fechado
    thickness = 5  # Espessura da linha do polígono
    pts = np.array(results[0])  # Converte resultados para array numpy

    return cv2.polylines(image, pts, isClosed, box_color, thickness)

def main():
    # Configurações de caminho e modelo
    image_path = "fiap1.png"
    model_path = "DB_TD500_resnet50.onnx"
    model_shape = (736, 736)  # Dimensões do modelo (largura, altura)

    # Inicializa o modelo de detecção de texto
    model = initialize_model(model_path, model_shape)

    # Pré-processa a imagem
    original_image, image, scaleWidth, scaleHeight =
preprocess_image(image_path, model_shape)

    # detecção de texto
    results = model.detect(image)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    # Ajusta as coordenadas dos resultados para a escala original da
imagem
    results = adjust_results(results, scaleWidth, scaleHeight)

    # Desenha os resultados na imagem original
    original_image = visualize(original_image, results)

    # Exibe a imagem com as detecções
    cv2.namedWindow("input", cv2.WINDOW_NORMAL)
    cv2.imshow("input", original_image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

Código-fonte 4 – ajustas as predições realizadas
Fonte: autor

Pronto! O resultado esperado é conseguir realizar a detecção dos textos das imagens, como demonstra a imagem a seguir:

Figura 6 – Exemplos de resultados de detecção de texto
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Reconhecimento de texto

Para realizar o reconhecimento do texto, vamos utilizar o modelo CRNN, que é uma rede neural pré-treinada para reconhecimento de texto. Vamos utilizar essa rede em conjunto com o modelo de detecção que criamos anteriormente, reaproveitando o nosso código e fazendo algumas alterações, incrementando algumas funções novas para o reconhecimento de texto.

Primeiramente, começamos realizando o download do modelo ‘crnn.onnx’. Fazemos isso a partir do link disponibilizado pela própria documentação da OpenCV, https://drive.google.com/uc?export=dowload&id=1ooaLR-rkTl8jdpGy1DoQs0-X0lQsB6Fj,como demonstrado na figura a seguir.

Repare que existem três versões disponíveis para download; para o nosso exemplo, vamos utilizar a primeira opção por ser a versão mais simples e leve para download. Esse modelo classifica 36 caracteres (0~9 + a~z). As outras versões podem ser utilizadas conforme a necessidade de desempenho para reconhecimento de mais caracteres, pontuações, símbolos etc.

Pretrained Models

TextRecognitionModel

```
crnn.onnx:
url: https://drive.google.com/uc?export=dowload&id=1ooaLR-rkTl8jdpGy1DoQs0-X0lQsB6Fj
sha: 270d92c9ccb670ada2459a25977e8deeaf8380d3,
alphabet_36.txt: https://drive.google.com/uc?export=dowload&id=1oPOYx5rQRp8L6XQciUwmwhMCfX0KyO4b
parameter setting: -rgb=0;
description: The classification number of this model is 36 (0~9 + a~z).
 The training dataset is MJSynth.

crnn_cs.onnx:
url: https://drive.google.com/uc?export=dowload&id=12diBsVJrS9ZEl6BNUiRp9s0xPALBS7kt
sha: a641e9c57a5147546f7a2dbea4fd322b47197cd5
alphabet_94.txt: https://drive.google.com/uc?export=dowload&id=1oKXxXKusquimp7XY1mFvj9nwLzldVgBR
parameter setting: -rgb=1;
description: The classification number of this model is 94 (0~9 + a~z + A~Z + punctuations).
 The training datasets are MJsynth and SynthText.

crnn_cs_CN.onnx:
url: https://drive.google.com/uc?export=dowload&id=1is4eYEUKH7HR7Gl37Sw4WPXx6Ir8oQEG
sha: 3940942b85761c7f240494cf662dcbf05dc00d14
alphabet_3944.txt: https://drive.google.com/uc?export=dowload&id=18IZUUdNzJ44heWTndDO6NNfIpJMmN-ul
parameter setting: -rgb=1;
description: The classification number of this model is 3944 (0~9 + a~z + A~Z + Chinese characters + special characters).
 The training dataset is ReCTS (https://rrc.cvc.uab.es/?ch=12).
```

More models can be found in here, which are taken from clovaai. You can train more models by CRNN, and convert models by `torch.onnx.export`.

Figura 7 – download modelo de reconhecimento CRNN
Fonte: autor

Começamos o nosso código carregando as bibliotecas e adaptando algumas funções. Como vamos trabalhar com dois modelos no mesmo código, vamos alterar

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

o nome da função ‘initialize_model’ para ‘initialize_textdetector_model’, dessa forma, temos certeza de que não vamos nos confundir.

```
import cv2
import numpy as np


def initialize_textdetector_model(model_path, model_shape,
binary_threshold=0.3, polygon_threshold=0.5,
                      max_candidates=200, unclip_ratio=2.0,
scale=1.0/255.0, mean=(122.67891434, 116.66876762, 104.00698793)):
    """
    Inicializa o modelo de detecção de texto.

    Parâmetros:
    - model_path: Caminho para o modelo ONNX.
    - model_shape: Tamanho da imagem de entrada (largura, altura).
    - binary_threshold: Limiar para a binarização.
    - polygon_threshold: Limiar para a formação do polígono.
    - max_candidates: Número máximo de candidatos (caixas de texto).
    - unclip_ratio: Fator de desenclipe para expandir as caixas.
    - scale: Fator de escala para normalização dos pixels.
    - mean: Média para normalização dos pixels.

    Retorna:
    - O modelo de detecção de texto inicializado.
    """

    # Carrega o modelo
    model = cv2.dnn_TextDetectionModel_DB(cv2.dnn.readNet(model_path))

    # Configurações do modelo
    model.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
    model.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
    model.setBinaryThreshold(binary_threshold)
    model.setPolygonThreshold(polygon_threshold)
    model.setUnclipRatio(unclip_ratio)
    model.setMaxCandidates(max_candidates)
    model.setInputParams(scale, model_shape, mean)

    return model
```

Código-fonte 5 – Detecta faces
Fonte: autor

O pré-processamento da imagem permanece o mesmo da função ‘preprocess_image’, que é responsável por carregar e redimensionar a imagem de

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

entrada para o tamanho adequado para a rede. Não é necessário realizar alterações na função ‘adjust_results’, que recebe as predições do método ‘detect’ que retorna uma lista de dois elementos, onde o elemento [0] contém os polígonos das regiões de texto, com as coordenadas normalizadas, e o elemento [1] contém o índice de confiança da predição.

A função ‘visualize’, que é responsável por desenhar os bounding boxes preditos, sofrerá algumas alterações. Além de desenhar as caixas delimitadoras, a função agora também irá exibir o texto que foi predito pela rede CRNN. A função recebe como parâmetro as caixas e os textos e itera sobre eles simultaneamente usando a função `zip`. Ou seja, para cada caixa `box` e texto `text`, os pontos da caixa são convertidos e desenhados na imagem.

```
def visualize(image, boxes, texts, color=(0, 255, 0), isClosed=True,
thickness=2):
    """
    Visualiza as caixas de texto e os textos reconhecidos na imagem.

    Parâmetros:
    - image: Imagem original.
    - boxes: Caixas de texto.
    - texts: Textos reconhecidos.
    - color: Cor das caixas.
    - isClosed: Indica se as caixas são fechadas.
    - thickness: Espessura das linhas das caixas.

    Retorna:
    - A imagem com as caixas e textos desenhados.
    """

   # Faz a iteração sobre as caixas e textos para desenhá-los na
imagem
    for box, text in zip(boxes[0], texts):
        points = np.array(box, np.int32).reshape((-1, 1, 2))
        image = cv2.polylines(image, [points], isClosed, color,
thickness)
        org = (int(box[1][0]), int(box[1][1]) ) # Define a posição do
texto
        cv2.putText(image, text, org, cv2.FONT_HERSHEY_SIMPLEX, 1.5,
(0, 255, 255), 2, cv2.LINE_AA)

    return image
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Código-fonte 6 – função visualize
Fonte: autor

Vamos criar a função ‘initialize_english_textrecognition_model`, que é responsável por inicializar um modelo CRNN. Essa função carrega o modelo, define as configurações necessárias e retorna algumas informações importantes para o processo de reconhecimento de texto. Vamos analisar alguns pontos:

O `character_set` é definido como '0123456789abcdefghijklmnopqrstuvwxyz'. Esse conjunto inclui todos os caracteres que o modelo pode reconhecer, abrangendo números e letras minúsculas do alfabeto inglês.

As `vertex_coordinates` são definidas como um array NumPy com as coordenadas dos quatro vértices para a transformação de perspectiva. Vamos usar para garantir que o texto esteja alinhado corretamente para o reconhecimento.

```
def initialize_english_textrecognition_model(model_path):
    """
    Inicializa o modelo de reconhecimento de texto.

    Parâmetros:
    - model_path: Caminho para o modelo ONNX.

    Retorna:
    - O modelo de reconhecimento de texto inicializado.
    - character_set: Conjunto de caracteres reconhecidos.
    - character_size: Tamanho da imagem de entrada para o
reconhecimento de texto.
    - vertex_coordinates: Coordenadas dos vértices para a
transformação de perspectiva.
    """

    # Carrega o modelo de reconhecimento de texto
    model = cv2.dnn.readNet(model_path)

    # Configurações do modelo
    model.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
    model.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)

    # Define o conjunto de caracteres reconhecidos
    character_set = '0123456789abcdefghijklmnopqrstuvwxyz'
    character_size = (100, 32)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    vertex_coordinates = np.array([
        [0, 31],
        [0, 0],
        [99, 0],
        [99, 31]
    ], dtype=np.float32)

    return model, character_set, character_size, vertex_coordinates
```

Código-fonte 7 – Função initialize_english_textrecognition_model
Fonte: autor

Com o modelo carregado, podemos avançar e criar a função que irá realizar o reconhecimento de texto. Vamos criar a função ‘recognize_text` para realizar essa tarefa. Podemos detalhar essa função em algumas partes:

- Começamos transformando as coordenadas da caixa delimitadora (bound box) para as coordenadas dos vértices e aplicamos a transformação de perspectiva para obter a região de texto. Em seguida, convertemos a imagem de colorida para tons de cinza e criamos um blob de imagem para o modelo de reconhecimento de texto. Por fim, fazemos a inferência do modelo e retornamos o texto reconhecido;

- A saída do modelo é interpretada para identificar os caracteres reconhecidos. A função np.argmax é usada para encontrar o índice do caractere mais provável em cada posição. Uma string de texto é construída a partir desses caracteres, ignorando caracteres nulos (c == 0) e substituindo-os por um hífen (-);

- Por fim, a string de texto `char_list` é refinada para remover caracteres repetidos consecutivos e hifens, resultando no texto final reconhecido.

```
def recognize_text(model, character_set, character_size,
vertex_coordinates, image, boxshape):
    """
    Reconhece o texto em uma região de texto.

    Parâmetros:
    - model: Modelo de reconhecimento de texto.
    - character_set: Conjunto de caracteres reconhecidos.
    - character_size: Tamanho da imagem de entrada para o
reconhecimento de texto.
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    - vertex_coordinates: Coordenadas dos vértices para a
transformação de perspectiva.
    - image: Imagem original.
    - boxshape: Forma da caixa de texto.

    Retorna:
    - O texto reconhecido.
    """

    vertices = boxshape.reshape((4, 2)).astype(np.float32)
    rotationMatrix = cv2.getPerspectiveTransform(vertices,
vertex_coordinates)
    cropped_image = cv2.warpPerspective(image, rotationMatrix,
character_size)
    cropped_image = cv2.cvtColor(cropped_image, cv2.COLOR_BGR2GRAY)
    text_blob = cv2.dnn.blobFromImage(cropped_image,
size=character_size, mean=127.5, scalefactor=1 / 127.5)

    model.setInput(text_blob)
    output_blob = model.forward()

    # Converte a saída do modelo para texto
    text = ''
    for i in range(output_blob.shape[0]):
        c = np.argmax(output_blob[i][0])
        if c != 0:
            text += character_set[c - 1]
        else:
            text += '-'
    print(text)
    char_list = []
    for i in range(len(text)):
        if text[i] != '-' and (not (i > 0 and text[i] == text[i -
1])):
            char_list.append(text[i])

    return ''.join(char_list)
```

Código-fonte 8 – Reconhece texto
Fonte: autor

Com todas as nossas funções já criadas e organizadas, podemos criar a função principal `main`, que irá orquestrar o nosso código chamando cada função de forma correta.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
def main():
    # Configurações de caminho e modelos
    image_path = "fiap1.png"
    model_shape = (736, 736)
    text_detector_model_path = "DB_TD500_resnet50.onnx"
    text_recognition_model_path = "crnn.onnx"


    # Inicializa o modelo de detecção de texto
    detector = initialize_textdetector_model(text_detector_model_path,
model_shape)

    # Inicializa o modelo de reconhecimento de texto
    recognizer, character_set, character_size, vertex_coordinates =
initialize_english_textrecognition_model(text_recognition_model_path)

    # Pré-processa a imagem de entrada
    original_image, image, scaleWidth, scaleHeight =
preprocess_image(image_path, model_shape)

    # Detecta as regiões de texto na imagem
    results = detector.detect(image)

    # Reconhece o texto nas regiões detectadas
    texts = []
    for box, score in zip(results[0], results[1]):
        text = recognize_text(recognizer, character_set,
character_size, vertex_coordinates, image, box.reshape(8))
        texts.append(text)

    # Alternativa para a iteração anterior usando list comprehension
    # texts = [recognize_text(recognizer, character_set,
character_size, vertex_coordinates, image, box.reshape(8)) for box,
score in zip(results[0], results[1])]

    # Ajusta as coordenadas dos resultados para a escala original da
imagem
    results = adjust_results(results, scaleWidth, scaleHeight)

    # Desenha os resultados na imagem original
    original_image = visualize(original_image, results, texts)

    cv2.namedWindow("input", cv2.WINDOW_NORMAL)
    cv2.imshow("input", original_image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == '__main__':
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    main()
```

Código-fonte 9 – Função main de reconhecimento de texto
Fonte: autor

Acabamos de detalhar todas as funções do nosso código. Esse é um código com muitas etapas e diversos detalhes. Por essa razão, apresentamos a seguir o código completo para garantir que nenhuma etapa seja perdida:

```
import cv2
import numpy as np

def initialize_textdetector_model(model_path, model_shape,
binary_threshold=0.3, polygon_threshold=0.5,
                      max_candidates=200, unclip_ratio=2.0,
scale=1.0/255.0, mean=(122.67891434, 116.66876762, 104.00698793)):
    """
    Inicializa o modelo de detecção de texto.

    Parâmetros:
    - model_path: Caminho para o modelo ONNX.
    - model_shape: Tamanho da imagem de entrada (largura, altura).
    - binary_threshold: Limiar para a binarização.
    - polygon_threshold: Limiar para a formação do polígono.
    - max_candidates: Número máximo de candidatos (caixas de texto).
    - unclip_ratio: Fator de desenclipe para expandir as caixas.
    - scale: Fator de escala para normalização dos pixels.
    - mean: Média para normalização dos pixels.

    Retorna:
    - O modelo de detecção de texto inicializado.
    """

    # Carrega o modelo
    model = cv2.dnn_TextDetectionModel_DB(cv2.dnn.readNet(model_path))

    # Configurações do modelo
    model.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
    model.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
    model.setBinaryThreshold(binary_threshold)
    model.setPolygonThreshold(polygon_threshold)
    model.setUnclipRatio(unclip_ratio)
    model.setMaxCandidates(max_candidates)
    model.setInputParams(scale, model_shape, mean)
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    return model

def initialize_english_textrecognition_model(model_path):
    """
    Inicializa o modelo de reconhecimento de texto.

    Parâmetros:
    - model_path: Caminho para o modelo ONNX.

    Retorna:
    - O modelo de reconhecimento de texto inicializado.
    - character_set: Conjunto de caracteres reconhecidos.
    - character_size: Tamanho da imagem de entrada para o
reconhecimento de texto.
    - vertex_coordinates: Coordenadas dos vértices para a
transformação de perspectiva.
    """

    # Carrega o modelo de reconhecimento de texto
    model = cv2.dnn.readNet(model_path)

    # Configurações do modelo
    model.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
    model.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)

    # Define o conjunto de caracteres reconhecidos
    # e o tamanho da imagem de entrada para o reconhecimento de texto
    # e as coordenadas dos vértices para a transformação de
perspectiva
    character_set = '0123456789abcdefghijklmnopqrstuvwxyz'
    character_size = (100, 32)
    vertex_coordinates = np.array([
        [0, 31],
        [0, 0],
        [99, 0],
        [99, 31]
    ], dtype=np.float32)

    return model, character_set, character_size, vertex_coordinates

def visualize(image, boxes, texts, color=(0, 255, 0), isClosed=True,
thickness=2):
    """
    Visualiza as caixas de texto e os textos reconhecidos na imagem.

    Parâmetros:
    - image: Imagem original.
    - boxes: Caixas de texto.
    - texts: Textos reconhecidos.
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    - color: Cor das caixas.
    - isClosed: Indica se as caixas são fechadas.
    - thickness: Espessura das linhas das caixas.

    Retorna:
    - A imagem com as caixas e textos desenhados.
    """

   # Faz a iteração sobre as caixas e textos para desenhá-los na
imagem
    for box, text in zip(boxes[0], texts):
        points = np.array(box, np.int32).reshape((-1, 1, 2))
        image = cv2.polylines(image, [points], isClosed, color,
thickness)
        org = (int(box[1][0]), int(box[1][1]) ) # Define a posição do
texto
        cv2.putText(image, text, org, cv2.FONT_HERSHEY_SIMPLEX, 1.5,
(0, 255, 255), 2, cv2.LINE_AA)
    return image

def recognize_text(model, character_set, character_size,
vertex_coordinates, image, boxshape):
    """
    Reconhece o texto em uma região de texto.

    Parâmetros:
    - model: Modelo de reconhecimento de texto.
    - character_set: Conjunto de caracteres reconhecidos.
    - character_size: Tamanho da imagem de entrada para o
reconhecimento de texto.
    - vertex_coordinates: Coordenadas dos vértices para a
transformação de perspectiva.
    - image: Imagem original.
    - boxshape: Forma da caixa de texto.

    Retorna:
    - O texto reconhecido.
    """

    # Transforma as coordenadas da caixa de texto
    # para as coordenadas dos vértices
    # e aplica a transformação de perspectiva
    # para obter a região de texto
    # em seguida, converte a imagem para tons de cinza
    # e cria um blob de imagem para o modelo
    # de reconhecimento de texto
    # por fim, faz a inferência do modelo
    # e retorna o texto reconhecido
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    vertices = boxshape.reshape((4, 2)).astype(np.float32)
    rotationMatrix = cv2.getPerspectiveTransform(vertices,
vertex_coordinates)
    cropped_image = cv2.warpPerspective(image, rotationMatrix,
character_size)
    cropped_image = cv2.cvtColor(cropped_image, cv2.COLOR_BGR2GRAY)
    text_blob = cv2.dnn.blobFromImage(cropped_image,
size=character_size, mean=127.5, scalefactor=1 / 127.5)

    model.setInput(text_blob)
    output_blob = model.forward()

    # Converte a saída do modelo para texto
    text = ''
    for i in range(output_blob.shape[0]):
        c = np.argmax(output_blob[i][0])
        if c != 0:
            text += character_set[c - 1]
        else:
            text += '-'
    print(text)
    char_list = []
    for i in range(len(text)):
        if text[i] != '-' and (not (i > 0 and text[i] == text[i -
1])):
            char_list.append(text[i])

    return ''.join(char_list)

def preprocess_image(image_path, model_shape):
    """
    Carrega e redimensiona a imagem de entrada para o tamanho do
modelo.

    Parâmetros:
    - image_path: Caminho para a imagem de entrada.
    - model_shape: Tamanho da imagem de entrada (largura, altura).

    Retorna:
    - original_image: A imagem original carregada.
    - image: A imagem redimensionada para o tamanho do modelo.
    - scaleWidth: Fator de escala para a largura.
    - scaleHeight: Fator de escala para a altura.
    """
    original_image = cv2.imread(image_path)
    original_h, original_w, _ = original_image.shape
    scaleHeight = original_h / model_shape[1]
    scaleWidth = original_w / model_shape[0]
    image = cv2.resize(original_image, model_shape)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    return original_image, image, scaleWidth, scaleHeight

def adjust_results(results, scaleWidth, scaleHeight):
    """
    Ajusta as coordenadas das caixas de resultados de volta para a
escala original da imagem.

    Parâmetros:
    - results: Resultados da detecção, contendo os polígonos das
regiões de texto, com as coordenadas normalizadas.
    - scaleWidth: Fator de escala para a largura.
    - scaleHeight: Fator de escala para a altura.

    Retorna:
    - results: Resultados ajustados para a escala original.
    """

    for i in range(len(results[0])):
        for j in range(4):
            box = results[0][i][j]
            results[0][i][j][0] = box[0] * scaleWidth
            results[0][i][j][1] = box[1] * scaleHeight
    return results

def main():
    # Configurações de caminho e modelos
    image_path = "fiap1.png"
    model_shape = (736, 736)
    text_detector_model_path = "DB_TD500_resnet50.onnx"
    text_recognition_model_path =
"text_recognition_CRNN_EN_2021sep.onnx"
    # text_recognition_model_path = "crnn.onnx"


    # Inicializa o modelo de detecção de texto
    detector = initialize_textdetector_model(text_detector_model_path,
model_shape)

    # Inicializa o modelo de reconhecimento de texto
    recognizer, character_set, character_size, vertex_coordinates =
initialize_english_textrecognition_model(text_recognition_model_path)

    # Pré-processa a imagem de entrada
    original_image, image, scaleWidth, scaleHeight =
preprocess_image(image_path, model_shape)

    # Detecta as regiões de texto na imagem
    results = detector.detect(image)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    # Reconhece o texto nas regiões detectadas
    texts = []
    for box, score in zip(results[0], results[1]):
        text = recognize_text(recognizer, character_set,
character_size, vertex_coordinates, image, box.reshape(8))
        texts.append(text)

    # Alternativa para a iteração anterior usando list comprehension
    # texts = [recognize_text(recognizer, character_set,
character_size, vertex_coordinates, image, box.reshape(8)) for box,
score in zip(results[0], results[1])]

    # Ajusta as coordenadas dos resultados para a escala original da
imagem
    results = adjust_results(results, scaleWidth, scaleHeight)

    # Desenha os resultados na imagem original
    original_image = visualize(original_image, results, texts)

    cv2.namedWindow("input", cv2.WINDOW_NORMAL)
    cv2.imshow("input", original_image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

Código-fonte 10 – Código completo de reconhecimento de texto
Fonte: autor

Pronto! O resultado esperado é conseguir realizar o reconhecimento dos textos das imagens, como demonstra a imagem a seguir.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 8 – Exemplos de resultados de reconhecimento de texto
Fonte: autor

Conseguimos compreender algumas das etapas para o reconhecimento de texto. No entanto, repare bem no resultado obtido: conseguimos realizar o reconhecimento do texto, mas algumas palavras estão sem os acentos e faltando espaços. Isso sugere que ainda há espaço para melhorias.

2.3 Conhecendo o PPOCR

O PPOCR é uma biblioteca de reconhecimento óptico de caracteres (OCR) desenvolvida pela Baidu como parte do PaddlePaddle, uma plataforma de deep learning de código aberto. O PPOCR fornece uma alternativa para detecção e reconhecimento de texto em imagens, com suporte para várias línguas, incluindo o português, além de se destacar por sua velocidade para detecção. Sua eficiência é atribuída à utilização de computação paralela e aceleração por GPU.

Além disso, o PaddleOCR é altamente adaptável e pode ser ajustado e afinado para tarefas e conjuntos de dados específicos, aumentando sua versatilidade e robustez em várias aplicações de OCR.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Dica: Para mais detalhes, acesse a publicação do artigo

PP-OCR: A Practical Ultra Lightweight OCR System - https://arxiv.org/pdf/2009.09941

De acordo com o artigo da publicação do PaddleOCR, a imagem a seguir demonstra a sua arquitetura, que segue uma abordagem similar a que utilizamos até o momento. A arquitetura é composta por três partes principais: detecção de texto, retificação das caixas detectadas e reconhecimento de texto.

1. Detecção de Texto: utiliza a Binarização Diferenciável (DB), um detector de texto eficiente baseado em uma rede de segmentação simples. Seis estratégias são aplicadas para melhorar a eficácia e eficiência do modelo, resultando em um tamanho de modelo de 1,4M;

2. Retificação das Caixas: as caixas de texto detectadas são transformadas em retângulos horizontais para facilitar o reconhecimento subsequente. Um classificador de direção é utilizado para garantir que o texto não esteja invertido;

3. Reconhecimento de Texto: utiliza CRNN (Convolutional Recurrent Neural Network), integrando extração de características e modelagem de sequência, com nove estratégias para melhorar a capacidade do modelo e reduzir seu tamanho, alcançando 1,6M para reconhecimento de caracteres chineses e em inglês e 900KB para símbolos alfanuméricos.

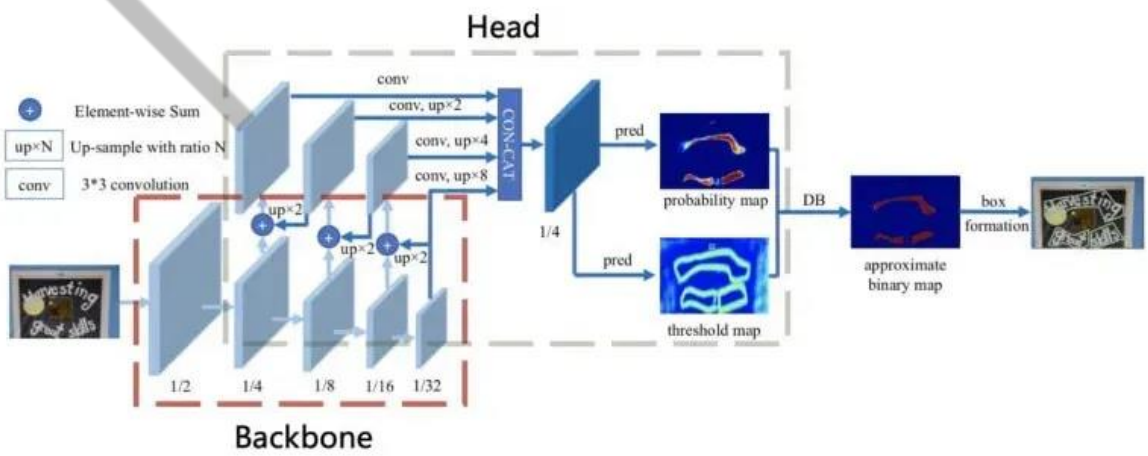

Figura 9 – Arquitetura de detecção PP-OCR
Fonte: Du (2020)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Para usar a PaddleOCR não é necessário retreinar, basta instalar a biblioteca via pip, que é o gerenciador de biblioteca do python. Vamos utilizar o Google Colab com o recurso de gpu ativado.

Para seu conhecimento: quando estamos trabalhando em notebooks, como Jupyter Notebooks ou Google Colab, temos a flexibilidade de executar comandos de terminal diretamente no ambiente do notebook.

Usamos o caractere '!' para executar instalações e outras operações de linha de comando sem precisar sair do ambiente do notebook.

```
# Instala PaddlePaddle GPU
!pip install paddlepaddle-gpu

# Instala o pacote PaddleOCR
!pip install -U pip
!pip install paddleocr
```

Código-fonte 11 – Infra PaddleOCR
Fonte: autor

Com a infraestrutura corretamente instalada, vamos abrir uma nova célula para o nosso código. Nessa célula, iremos instanciar um objeto PaddleOCR e chamar o método ocr, passando o caminho da imagem. O retorno é uma lista de listas contendo as coordenadas dos bounding boxes e uma tupla com os textos preditos e o índice de confiança.

```
from paddleocr import PaddleOCR
from matplotlib import pyplot as plt
import cv2
import os
import numpy as np

def process_results(results):
    """Processa os resultados da OCR para obter caixas de texto,
textos e pontuações."""
    boxes = [line[0] for line in results[0]]
    txts = [line[1][0] for line in results[0]]
    scores = [line[1][1] for line in results[0]]
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    return boxes, txts, scores

def annotate_image(image, boxes, txts):
    """Desenha os polígonos e textos na imagem."""
    for box, txt in zip(boxes, txts):
        points = np.array(box, np.int32).reshape((-1, 1, 2))
        cv2.polylines(image, [points], isClosed=True, color=(0, 255,
0), thickness=2)
        org = (int(box[0][0]), int(box[0][1]) - 10)
        cv2.putText(image, txt, org, cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(0, 255, 0), 2, cv2.LINE_AA)
    return image

def display_image(image):
    """Converte a imagem para RGB e exibe-a."""
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(10, 10))
    plt.imshow(image_rgb)
    plt.axis('off')
    plt.show()

def main(img_path):
    """Função principal que executa o pipeline de OCR."""

    # Inicializa o OCR
    ocr = PaddleOCR(use_angle_cls=True, lang='en')

    # Realiza OCR na imagem
    results = ocr.ocr(img_path, cls=True)

    # Processa os resultados
    boxes, txts, scores = process_results(results)

    # Desenha as predições na imagem
    image = cv2.imread(img_path)
    annotated_image = annotate_image(image, boxes, txts)

    # Exibe a imagem com as predições
    display_image(annotated_image)

if __name__ == '__main__':
    img_path = 'fiap.png'
    main(img_path)
```

Código-fonte 12 – Reconhecimento OCR com PaddleOCR
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Se tudo ocorreu bem, o resultado esperado deve ser semelhante ao da imagem a seguir, onde conseguimos, com uma boa acurácia, realizar ao reconhecimento de caracteres da imagem.

Figura 10 – Reconhecimento de texto PaddleOCR
Fonte: autor

O PaddleOCR oferece suporte a diversos idiomas, incluindo português. Esse recurso é particularmente interessante para o reconhecimento de caracteres especiais que não são nativos do inglês, que é o idioma padrão. No caso do português, isso inclui a capacidade de reconhecer corretamente acentos e outras marcas diacríticas, garantindo uma precisão maior no reconhecimento de texto em documentos e imagens nesse idioma. Para isso, basta ajustar de lang=’en’ lang=’pt’ na inicialização do modelo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 GENERATIVE IA EM OCR

Tradicionalmente, métodos baseados em redes neurais convolucionais (CNNs) e redes recorrentes (RNNs) têm sido usados para essa tarefa. Recentemente, modelos baseados em Transformers, uma classe de modelos de IA generativa, têm surgido como uma alternativa poderosa, oferecendo vantagens em precisão e flexibilidade. Vamos explorar como Transformers são aplicados no OCR, suas vantagens e implementações práticas.

Figura 11 – Optimus Prime e Bumble Bee; Transformers na Universal Studios
Fonte: prayitno (2012) https://commons.wikimedia.org/w/index.php?curid=22523905

Mas afinal, o que são Transformers? Já vou avisando que não têm relação nenhuma com os robôs Autobots e Decepticons dos filmes, que mudam de forma. Brincadeiras à parte, os Transformers são uma classe de modelos de aprendizado profundo, amplamente utilizados em IA generativa, que utilizam mecanismos de atenção para processar dados sequenciais. Esse modelo vem modificando o processamento de linguagem natural (NLP) e, mais recentemente, tem sido aplicado com sucesso em várias tarefas de visão computacional, incluindo OCR.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O núcleo dos Transformers é o mecanismo de atenção, que permite ao modelo focar em diferentes partes da entrada de forma adaptativa. Eles foram introduzidos em 2017 pelos pesquisadores da Google em um artigo chamado "Attention is All You Need". Existem diversas implementações práticas de Transformers no contexto de OCR. Algumas delas são:

- Vision Transformers (ViT): embora não seja especificamente projetado para OCR, o Vision Transformer tem sido adaptado para essa tarefa com sucesso. Ele divide a imagem em patches e aplica um Transformer a esses patches para realizar a classificação e o reconhecimento de caracteres;

- TrOCR (Transformer OCR): é uma arquitetura de transformer end-to-end desenvolvida para OCR. Utiliza uma combinação de codificadores e decodificadores de transformers para detectar e reconhecer texto diretamente a partir de imagens.

3.1 Vision Transformers

Vision Transformers (ViTs) permite realizar tarefas visuais convencionais, como classificação, segmentação e geração de legendas de imagem para texto. Um dos modelos mais recentes e promissores é o CoCa, utilizando a implementação OpenCLIP.

Saiba mais em:

CoCa: Contrastive Captioners are Image-Text Foundation Models
https://arxiv.org/pdf/2205.01917.pdf.

Clip, Learning Transferable Visual Models From Natural Language Supervision
https://arxiv.org/abs/2103.00020

CoCa é um modelo que integra capacidades de reconhecimento de texto e geração de contexto a partir de imagens. Por sua vez, o OpenCLIP é uma biblioteca open source que facilita o uso de modelos CLIP (Contrastive Language-Image Pre-Training) pré-treinados, desenvolvida pela OpenAI. O CLIP, publicado no artigo "Learning Transferable Visual Models From Natural Language Supervision" por Radford et al. (2021), é uma rede neural pré-treinada em um enorme conjunto de

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

dados de pares imagem-texto. A abordagem de aprendizagem contrastiva pode mapear imagens e texto para um espaço de incorporação comum. Isso permite que o CLIP execute uma variedade de tarefas, como classificação de imagem zero-shot, recuperação de imagem e geração de texto para imagem.

A imagem a seguir mostra um diagrama do modelo CLIP que inclui dois componentes principais: um modelo de texto (Text Encoder) e um modelo de imagem (Image Encoder).

- Text Encoder: utiliza um modelo de Transformer para processar textos. Ele converte uma descrição textual em um vetor de embedding. Por exemplo, "Pepper the aussie pup", o texto é convertido em um vetor de embedding, representado como T1, T2, ..., TN;

- Image Encoder: pode utilizar Vision Transformers (ViT) ou CNNs. Quando configurado com Vision Transformers, o modelo divide uma imagem em patches, que são processados como sequências, similar ao processamento de palavras em textos pelos Transformers. Se configurado com CNNs, utiliza uma arquitetura tradicional de rede neural convolucional. A imagem é convertida em um vetor de embedding, representado como I1, I2, ..., IN.

A similaridade entre os embeddings é calculada para determinar quão bem cada par de texto e imagem corresponde. Isso é ilustrado pela matriz onde cada célula (por exemplo, I1T1) representa a similaridade entre um embedding de imagem e um embedding de texto.

Por fim, quando uma nova imagem é fornecida, a classificação zero-shot é realizada e seu embedding é comparado com os embeddings das descrições de classes. O modelo determina a classe mais provável da imagem com base na similaridade das representações de embedding. No exemplo, a descrição "A photo of a [object]" é embutida e comparada com o embedding da nova imagem para prever a classe correta.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

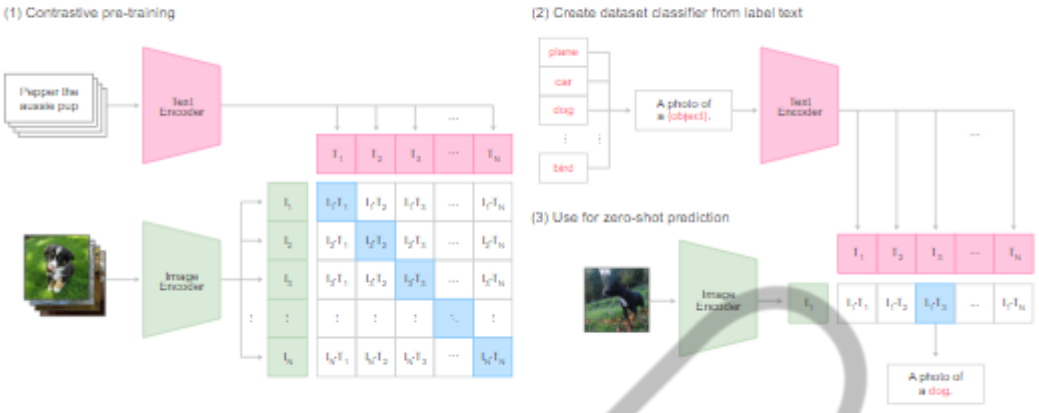

Figura 12 – Arquitetura open Clip
Fonte: https://github.com/openai/CLIP

Vamos utilizar esse modelo para fazer o reconhecimento de texto a partir de uma imagem por meio biblioteca Transformers, para isso, utilizaremos o Google Colab.

Começamos fazendo a instalação das bibliotecas openclip e transformers diretamente de uma célula do notebook.

```
### Instalação das bibliotecas necessárias


!pip install open_clip_torch transformers==4.38.2
```

Código-fonte 13 – Instala biblioteca openclip e transformers
Fonte: autor

Com as bibliotecas instaladas, podemos carregar o modelo que vamos utilizar; o método list_pretrained exibe todos os modelos disponíveis para uso.

```
import open_clip
open_clip.list_pretrained()
```

Código-fonte 14 – list pretrained

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: autor

A imagem a seguir exibe alguns dos modelos disponíveis.

```
('ViT-L-14', 'commonpool_xl_s13b_b90k'),
('ViT-L-14-quickgelu', 'metaclip_400m'),
('ViT-L-14-quickgelu', 'metaclip_fullcc'),
('ViT-L-14-quickgelu', 'dfn2b'),
('ViT-L-14-336', 'openai'),
('ViT-H-14', 'laion2b_s32b_b79k'),
('ViT-H-14-quickgelu', 'metaclip_fullcc'),
('ViT-H-14-quickgelu', 'dfn5b'),
('ViT-H-14-378-quickgelu', 'dfn5b'),
('ViT-g-14', 'laion2b_s12b_b42k'),
('ViT-g-14', 'laion2b_s34b_b88k'),
('ViT-bigG-14', 'laion2b_s39b_b160k'),
('roberta-ViT-B-32', 'laion2b_s12b_b32k'),
('xlm-roberta-base-ViT-B-32', 'laion5b_s13b_b90k'),
('xlm-roberta-large-ViT-H-14', 'frozen_laion5b_s13b_b90k'),
('convnext_base', 'laion400m_s13b_b51k'),
('convnext_base_w', 'laion2b_s13b_b82k'),
('convnext_base_w', 'laion2b_s13b_b82k_augreg'),
('convnext_base_w', 'laion_aesthetic_s13b_b82k'),
('convnext_base_w_320', 'laion_aesthetic_s13b_b82k'),
('convnext_base_w_320', 'laion_aesthetic_s13b_b82k_augreg'),
('convnext_large_d', 'laion2b_s26b_b102k_augreg'),
('convnext_large_d_320', 'laion2b_s29b_b131k_ft'),
('convnext_large_d_320', 'laion2b_s29b_b131k_ft_soup'),
('convnext_xxlarge', 'laion2b_s34b_b82k_augreg'),
('convnext_xxlarge', 'laion2b_s34b_b82k_augreg_rewind'),
('convnext_xxlarge', 'laion2b_s34b_b82k_augreg_soup'),
('coca_ViT-B-32', 'laion2b_s13b_b90k'),
('coca_ViT-B-32', 'mscoco_finetuned_laion2b_s13b_b90k'),
('coca_ViT-L-14', 'laion2b_s13b_b90k'),
('coca_ViT-L-14', 'mscoco_finetuned_laion2b_s13b_b90k'),
('EVA01-g-14', 'laion400m_s11b_b41k'),
('EVA01-g-14-plus', 'merged2b_s11b_b114k'),
('EVA02-B-16', 'merged2b_s8b_b131k'),
('EVA02-L-14', 'merged2b_s4b_b131k'),
('EVA02-L-14-336', 'merged2b_s6b_b61k'),
('EVA02-E-14', 'laion2b_s4b_b115k'),
('EVA02-E-14-plus', 'laion2b_s9b_b144k'),
('ViT-B-16-SigLIP', 'webli'),
```

Figura 13 – lista de modelos pré-treinados
Fonte: autor

Vamos utilizar a função create_model_and_transforms da biblioteca open_clip para carregar o modelo coca_ViT-L-14 pré-treinado no dataset laion2b_s13b_b90k.

```
import open_clip

model, _, transform = open_clip.create_model_and_transforms(
    model_name="coca_ViT-L-14",
  pretrained="laion2b_s13b_b90k"
)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Código-fonte 15 – Carrega modelo
Fonte: autor

Após o download do modelo, vamos poder utilizá-lo, contudo, é necessário carregar e pré-processar a imagem antes de realizar a predição. Pra realizar essa tarefa, vamos usar a biblioteca PIL (Python Imaging Library) para abrir uma imagem, convertê-la para o formato RGB e aplicar uma transformação definida pelo transform. Em seguida, o método .unsqueeze(0) adiciona uma dimensão extra no início do tensor, convertendo-o de uma imagem de 3 dimensões (altura, largura, canais) para um tensor de 4 dimensões (batch size, canais, altura, largura). Esse formato é necessário para que a imagem possa ser processada em lotes pelo modelo.

```
from PIL import Image

im = Image.open("camiseta.jpg").convert("RGB")


im = transform(im).unsqueeze(0)
```

Código-fonte 16 – Carrega imagem
Fonte: autor

O método .generate é responsável por gerar os tokens de uma legenda para a imagem processada, usando o modelo coca_ViT-L-14 pré-treinado. Após a geração dos tokens, eles são decodificados de volta para texto legível com o médoto .decode da openclip. Essa operação envolve a remoção dos tokens especiais <start_of_text> e <end_of_text>, além da junção dos tokens em uma string.

```
generated = model.generate(im)

caption =
open_clip.decode(generated[0]).split("<end_of_text>")[0].replace("<sta
rt_of_text>", "")

print(caption)
```

Código-fonte 17 – gera predição de texto
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Agora que já compreendemos cada etapa do processo, podemos incrementar um pouco nosso código para realizar um sistema que, além de descrever o que está escrito na imagem, também retorna o tipo de objeto. Vamos pensar no caso de uma loja de e-commerce; pode ser interessante saber informações do produto de forma automática.

```
from PIL import Image
import requests
from io import BytesIO
import torch.nn.functional as F
import open_clip
open_clip.list_pretrained()

model, _, transform = open_clip.create_model_and_transforms(
    model_name="coca_ViT-L-14",
  pretrained="laion2b_s13b_b90k"
)


def get_caption(link):
    print(link)
    response = requests.get(link)
    img = Image.open(BytesIO(response.content))
    im = transform(img).unsqueeze(0)
    generated = model.generate(im)
    caption =
open_clip.decode(generated[0]).split("<end_of_text>")[0].replace("<sta
rt_of_text>", "")
    print(caption)
    dimension = (512,512)
    display(img.resize(dimension))


links =
['https://i.etsystatic.com/34225104/r/il/4a6846/4993229531/il_1140xN.4
993229531_bddx.jpg',

'https://i.etsystatic.com/25056868/r/il/824467/3970311611/il_1140xN.39
70311611_j4ja.jpg',

'https://i.etsystatic.com/31984822/r/il/3c67e2/5245999034/il_1140xN.52
45999034_ojxa.jpg',

'https://i.etsystatic.com/5768774/r/il/20f8cb/3397452626/il_1140xN.339
7452626_3fz4.jpg'
]
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
for link in links:
  get_caption(link)
```

Código-fonte 18 – lista de camisetas
Fonte: autor

O resultado esperado é conseguir extrair o texto das imagens e saber qual o objeto que se trata.

https://i.etsystatic.com/34225104/r/il/4a6846/4993229531/il_1140xN.4993229531_bddx.jpg
jesus made me a fisher and a man i catch me cleans t - shirt

Figura 14 – exemplo de resultado de OCR com Transformers
Fonte: autor

Assim como todo e qualquer modelo de IA Generativo, é importante lembrar o que o modelo pode alucinar e errar a sua predição. Para evitar isso, é possível realizar etapas de ajustes fino ao modelo pré-treinado.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3.2 TrOCR

A arquitetura TrOCR (Transformer-based Optical Character Recognition) é um modelo avançado de OCR que combina técnicas de visão computacional e processamento de linguagem natural utilizando a arquitetura Transformer. O objetivo do TrOCR é converter imagens de texto impresso ou manuscrito em texto editável, aproveitando as vantagens dos Transformers em capturar dependências de longo alcance e contextuais.

Saiba mais em:

- TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models https://arxiv.org/abs/2109.10282

A figura a seguir ilustra a arquitetura do TrOCR, que é composta por dois componentes principais: o Encoder (codificador) e o Decoder (decodificador). Vamos detalhar cada parte do diagrama para entender o seu funcionamento.

Entrada e Patches:

- Input Image: a imagem de entrada contém texto que precisa ser reconhecido. No exemplo, a imagem mostra "LICENSEE OF MCDONALD'S";

- Image Patches: a imagem de entrada é dividida em pequenos blocos ou patches. Cada um desses patches contém uma pequena parte da imagem original. No exemplo, os patches conter partes de letras como "I", "NS", "E" etc.;

- Flatten: os patches da imagem são achatados em vetores, formando uma sequência linear de patches;

- Patch Embedding: cada patch é convertido em um vetor de alta dimensionalidade. Esses vetores contêm informações sobre o conteúdo visual dos patches;

- Position Embedding: são adicionados embeddings de posição para cada patch, ajudando o modelo a manter a informação sobre a localização de cada patch na imagem original.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Codificador (Encoder):

- Multi-Head Attention: essa camada de Transforme permite que o modelo foque em diferentes partes dos patches de imagem simultaneamente;

- Feed Forward: essa camada processa as saídas da camada de atenção, aplicando transformações não-lineares;

- Essas camadas são repetidas N vezes (indicadas como x N no diagrama);

- Encoder Outputs: as saídas do encoder são vetores que representam a imagem processada em um formato que pode ser utilizado pelo decodificador para gerar texto.

Decodificador (Decoder)

- Masked Multi-Head Attention: essa camada de transforme permite que o decoder gere a sequência de texto de forma sequencial, prevenindo que ele veja tokens futuros durante o treinamento;

- Multi-Head Attention e Feed Forward: essas camadas funcionam de maneira similar as do codificador, processando as entradas e gerando a saída de texto;

- Essas camadas também são repetidas N vezes (indicadas como x N no diagrama).

Saída do Texto

- Outputs: a saída do modelo é a sequência de texto reconhecida a partir da imagem. No exemplo, a sequência de texto é "LICENSEE OF MCDONALD'S".

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

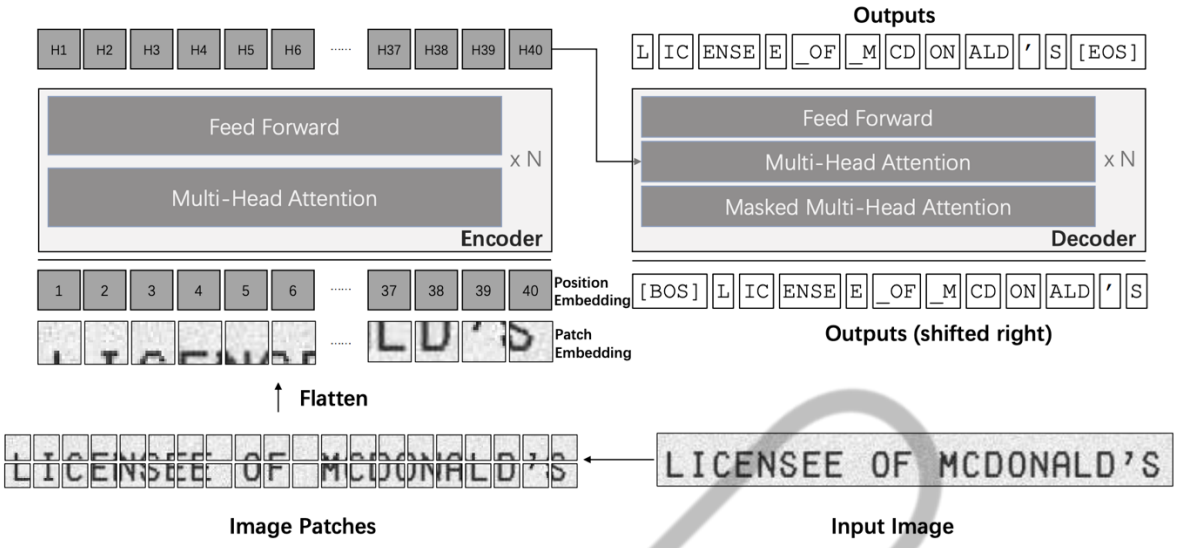

Figura 15 – Arquitetura TrOCR
Fonte: Li (2021)

Agora que compreendemos o funcionamento dessa arquitetura, vamos utilizar esse modelo para fazer o reconhecimento de texto utilizando o Google Colab. Começamos fazendo a instalação das bibliotecas diretamente de uma célula do notebook.

```
!pip install torch torchvision transformers sentencepiece
```

Código-fonte 19 – instala bibliotecas
Fonte: autor

Com as bibliotecas instaladas, podemos realizar o carregamento das bibliotecas e importar o modelo TrOCR, da Huggingface. A função principal, main, coordena o fluxo de execução, começando pela configuração do dispositivo (CPU ou GPU, pois estamos utilizando o pytorch) e o carregamento do modelo e processador pré-treinados, como o IAM Handwritten e o SROIE printed receipts.

O dataset IAM Handwritten contém imagens de texto manuscrito, e o ajuste fino nesse dataset melhora a capacidade do modelo de reconhecer texto manuscrito. O dataset SROIE, por sua vez, reconhece texto impresso. Existem três escalas de modelos tanto para manuscrito quanto para impresso – Small, Base e Large. Vamos utilizar os modelos hospedados no Hugging Face.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Em seguida, a função process_images é chamada para processar uma lista de URLs de imagens. Para cada URL, a função download_image baixa a imagem e a converte para o formato adequado. A função ocr realiza o OCR na imagem, utilizando o modelo e o processador para converter a imagem em texto.

Finalmente, a função display_image_with_text exibe a imagem com o texto OCR resultante como título, utilizando a biblioteca matplotlib para visualização. Esse fluxo modularizado permite a fácil extensão e manutenção do código, além de tornar o processo claro e didático.

```
from transformers import TrOCRProcessor, VisionEncoderDecoderModel
from PIL import Image
from tqdm.auto import tqdm
import requests
from io import BytesIO
import matplotlib.pyplot as plt
import torch

def download_image(url):
    """
    Faz o download de uma imagem de uma URL e a retorna como uma PIL
Image.

    :param url: String, URL da imagem.
    :return: PIL Image, imagem baixada.
    """
    response = requests.get(url)
    image = Image.open(BytesIO(response.content)).convert('RGB')
    return image

def ocr(image, processor, model, device):
    """
    Realiza OCR (reconhecimento óptico de caracteres) em uma imagem
usando um modelo Huggingface.

    :param image: PIL Image, imagem de entrada.
    :param processor: Huggingface OCR processor.
    :param model: Huggingface OCR model.
    :param device: PyTorch device, dispositivo para execução do modelo
(CPU ou GPU).
    :return: String, texto OCR'd da imagem.
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    """
    pixel_values = processor(image,
return_tensors='pt').pixel_values.to(device)
    generated_ids = model.generate(pixel_values)
    generated_text = processor.batch_decode(generated_ids,
skip_special_tokens=True)[0]
    return generated_text

def display_image_with_text(image, text):
    """
    Exibe a imagem com o texto OCR resultante como título.

    :param image: PIL Image, imagem de entrada.
    :param text: String, texto OCR resultante.
    """
    plt.figure(figsize=(7, 4))
    plt.imshow(image)
    plt.title(text)
    plt.axis('off')
    plt.show()

def process_images(urls, num_samples, processor, model, device):
    """
    Processa uma lista de URLs de imagens, realiza OCR e exibe os
resultados.

    :param urls: List, lista de URLs das imagens de entrada.
    :param num_samples: Int, número de amostras para avaliar.
    :param processor: Huggingface OCR processor.
    :param model: Huggingface OCR model.
    :param device: PyTorch device, dispositivo para execução do modelo
(CPU ou GPU).
    """
    for i, url in tqdm(enumerate(urls), total=min(num_samples,
len(urls))):
        if i == num_samples:
            break
        image = download_image(url)
        text = ocr(image, processor, model, device)
        display_image_with_text(image, text)

def main(urls, num_samples=4):
    """
    Função principal que executa o pipeline de OCR.

    :param urls: List, lista de URLs das imagens de entrada.
    :param num_samples: Int, número de amostras para avaliar.
    """
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    # Verifica e configura o dispositivo para usar GPU se disponível,
caso contrário, usa CPU
    device = torch.device('cuda:0' if torch.cuda.is_available() else
'cpu')

    # Carrega o modelo e o processador para texto impresso
    processor = TrOCRProcessor.from_pretrained('microsoft/trocr-small-
printed')
    model =
VisionEncoderDecoderModel.from_pretrained('microsoft/trocr-small-
printed').to(device)

    # Processa as imagens e realiza OCR
    process_images(urls, num_samples, processor, model, device)

if __name__ == '__main__':
    # Lista de URLs das imagens
    links = [
        'https://i.postimg.cc/ZKwLg2Gw/367-14.png',
        'https://fki.tic.heia-fr.ch/static/img/a01-122-02-00.jpg',

'https://www.tnb.studio/storage/blog/TNB_Blog_Jul_14.07_A.jpg',

'https://i.etsystatic.com/5768774/r/il/20f8cb/3397452626/il_1140xN.339
7452626_3fz4.jpg'
    ]

    main(links)
```

Código-fonte 20 – Código TrOCR
Fonte: autor

O resultado esperado é o reconhecimento do texto para os dois primeiros exemplos e o reconhecimento falso para o terceiro link de exemplo, como mostra a figura a seguir.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 16 – Reconhecimento TrOCR Hugging Face
Fonte: autor

Uma vantagem desse modelo é que ele foi treinado com diversos conjuntos de dados para aplicações distintas, como texto impresso, manuscrito, documentos comerciais, científicos e textos embutidos em cenas naturais. A combinação desses datasets durante o treinamento permite que o TrOCR tenha uma melhor generalização e robustez ao lidar com diferentes tipos de texto e contextos visuais. As opções disponíveis podem ser encontradas diretamente no site do hungging Face no link https://huggingface.co/models?search=microsoft/trocr, como mostra a figura a seguir.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

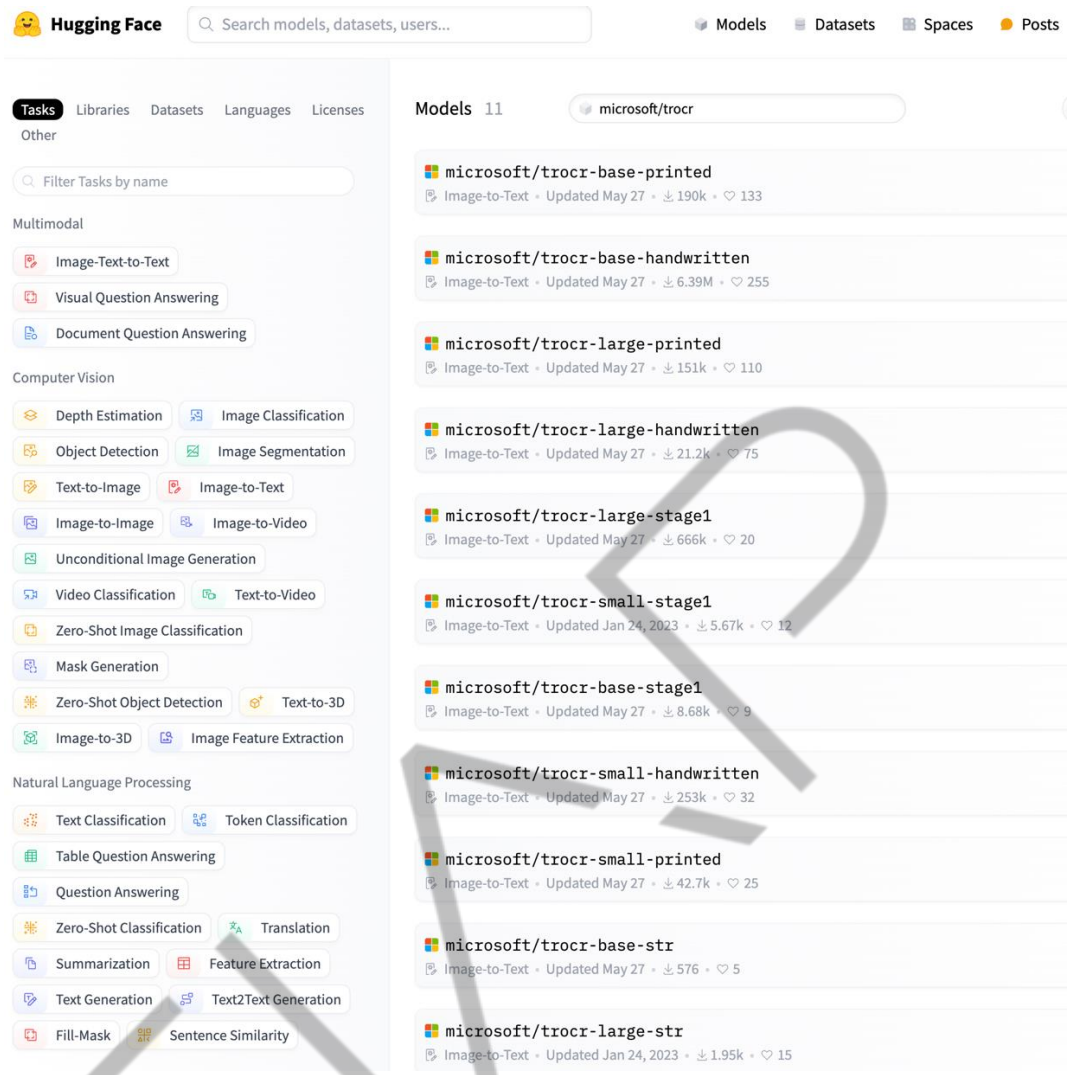

Figura 17 – Hugging Face - TrOCR
Fonte: autor

Contudo, possui limitações como todos, nesse caso, ele funciona melhor com textos por linha ou recorte de texto, sendo necessário uma etapa de pré-processamento ou treinamento do modelo para que funcione melhor.

Ao longo dos nossos estudos, conhecemos diversos modelos, mas tenha em mente que existem outros que não exploramos nesse capítulo, tais como DocTr, EasyOCR e Tesseract. Esses modelos possuem fácil integração com Python e são simples de serem utilizados em diversas aplicações.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4 HANDS ON: PRATICANDO O RECONHECIMENTO DE CARACTERES DE TEXTO

4.1 Aquecimento

Neste capítulo, vimos diversas técnicas avançadas de deep learning e como são aplicadas em problemas de visão computacional para reconhecimento de texto. Antes de começar o projeto deste capítulo, certifique-se de ter acesso ao Google Colab e que o ambiente está ativo para uso da GPU. Isso será particularmente útil na etapa de execução do modelo.

4.2 Seu desafio

Sua tarefa é criar um modelo de reconhecimento óptico de caracteres (OCR) que seja capaz de predizer o texto presente em jornais antigos. Você deve:

- Usar a seguinte notícia do link: https://s2-oglobo.glbimg.com/YqlCdLPRU7k0EjZhO53P0z4hpVU=/0x0:749x449/1000x0/smart/filters:strip_icc()/i.s3.glbimg.com/v1/AUTH_da025474c0c44edd99332dddb09cabe8/internal_photos/bs/2023/a/8/d9g5BMSmWB3b50wEFFPw/ayrton-senna-5.jpg

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

AFP

A equipe médica, em desespero, transporta o corpo de Senna na maca para o helicóptero que o levou a Bolonha

Corpo do piloto pode chegar hoje ao Brasil

Na longa espera, a realidade vence a fé

BOLONHA — O clima era de imensa tristeza e ansiedade. Havia um silêncio pesado no Hospital Maggiore, os olhos dos presentes estavam cheios de lágrimas e a tensão era evidente. Havia também esperança de uma melhora no quadro geral, quando surgiu outro depoimento explosivo: do anestesista Conso, que atendera Ayrton Senna na pista, logo após a batida.

— Desde o primeiro momento vimos que ele corria risco de vida. Havia sofrido fraturas no crânio, com perda da massa cerebral. E perdia muito sangue. O pulso havia desaparecido e só voltou graças ao trabalho dos médicos. A cabeça foi muito atingida.

Então, as esperanças sobre uma possível recuperação de Ayrton Senna se esfumaçaram um minuto depois, quando a doutora Maria Teresa Fiandri foi clara sobre as reais possibilidades do piloto brasileiro em termos de percentagem. Apertando os lábios, comentou:

— Acho que estamos na dependência de um milagre.

Figura 18 – Notícia jornal - Senna
Fonte: O globo (2023)

- Aplicar as técnicas de pré-processamento de imagens;

- Definir a estratégia para detecção e reconhecimento;

- Carregar e executar o modelo usando GPU no Google Colab;

- Realizar o OCR na imagem fornecida

É esperado que o script seja capaz de informar com precisão a maior parte do texto escrito na notícia do jornal.

4.3 Dicas de solução

A proposta é que você aplique os conceitos aprendidos sobre OCR de maneira prática, construindo um sistema robusto que demonstre sua capacidade de reconhecer texto na imagem proposta. Aqui estão algumas dicas para ajudá-lo a resolver o desafio:

- Pré-processamento de Imagens: aplique técnicas de pré-processamento como recorte, contraste e redimensionamento para melhorar a qualidade das imagens antes de passá-las pelo modelo OCR. Separe a imagem em algumas partes para facilitar o reconhecimento;.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Teste os modelos: aplique os diversos modelos de OCR estudados e as variações deles. O seu pipeline pode incluir validação dos modelos de detecção do texto e modelos de reconhecimento de texto. modelos de aprendizado profundo baseados em CNN ou Transformers;

- Avaliação de Resultados: realize a avaliação e a precisão visual exibindo o resultado para saber validar qual modelo se aplica melhor à imagem de teste e identificar áreas para melhorias.

Bons estudos!

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

Du, Y. et al. **PP-OCR:** A Practical Ultra Lightweight OCR System. 2020. Disponível em <https://arxiv.org/pdf/2009.09941>. Acesso em: 18 jul. 2024.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. Deep learning. MIT press, 2016.

HAYKIN, Simon. **Redes neurais:** princípios e prática. Bookman Editora, 2008.

LI, M. et al. **TrOCR:** Transformer-based Optical Character Recognition with Pre-trained Models. 2021. Disponível em <https://arxiv.org/pdf/2109.10282>. Acesso em: 18 jul. 2024.

LUGER, George F. Inteligência Artificial, 6ª ed. São Paulo: Pearson Education do Brasil, 2013 (biblioteca virtual).

MISHRA, Ranjan Kumar; REDDY, GY Sandesh; PATHAK, Himanshu. **The understanding of deep learning:** A comprehensive review. Mathematical Problems in Engineering, 2021, 2021: 1-15.

MITCHELL, Tom M. Machine Learning. 1st edition, New York, United States: McGraw-Hill Education, 1997.

O GLOBO. **Morte de Ayrton Senna faz 29 anos:** Relembre trajetória do piloto que virou patrono do esporte brasileiro. 2023. Disponível em <https://oglobo.globo.com/esportes/noticia/2023/05/morte-de-ayrton-senna-faz-29-anos-relembre-trajetoria-do-piloto-que-virou-patrono-do-esporte-brasileiro.ghtml>. Acesso em: 18 jul. 2024.

RADFORD, A. et al. **Learning Transferable Visual Models From Natural Language Supervision**. 2021. Disponível em <https://arxiv.org/abs/2103.00020>. Acesso em: 18 jul. 2024.

RUSSEL, Stewart; NORVIG, Peter. Inteligência artificial. 3ª. Ed., Rio de Janeiro: Campus, 2012.

SHI, B.; BAI, X.; YAO, C. **An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition**. 2015. Disponível em: <https://arxiv.org/pdf/1507.05717>. Acesso em: 18 de jun. 2024.

VASWANI, A. el al. **Attention is all you need**. *Advances in Neural Information Processing Systems*, *30*, 5998-6008. 2017. Disponível em: <https://arxiv.org/abs/1706.03762>. Acesso em: 18 jul. 2024.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

WANI, M. Arif; KHOSHGOFTAAR, Taghi M.; PALADE, Vasile (ed.). Deep Learning Applications, Volume 3. Springer, 2021.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com